

INPUT AND OUTPUT

Stream responses in real-time

Get real-time responses from the Agent SDK as text and tool calls stream in

By default, the Agent SDK yields complete `AssistantMessage` objects after Claude finishes generating each response. To receive incremental updates as text and tool calls are generated, enable partial message streaming by setting `include_partial_messages` (Python) or `includePartialMessages` (TypeScript) to `true` in your options.

 This page covers output streaming (receiving tokens in real-time). For input modes (how you send messages), see [Send messages to agents](#). You can also [stream responses using the Agent SDK via the CLI](#).

Enable streaming output

To enable streaming, set `include_partial_messages` (Python) or `includePartialMessages` (TypeScript) to `true` in your options. This causes the SDK to yield `StreamEvent` messages containing raw API events as they arrive, in addition to the usual `AssistantMessage` and `ResultMessage`.

Your code then needs to:

1. Check each message's type to distinguish `StreamEvent` from other message types
2. For `StreamEvent`, extract the `event` field and check its `type`
3. Look for `content_block_delta` events where `delta.type` is `text_delta`, which contain the actual text chunks

The example below enables streaming and prints text chunks as they arrive. Notice the nested type checks: first for `StreamEvent`, then for `content_block_delta`, then for `text_delta`:

```

from claude_agent_sdk import query, ClaudeAgentOptions
from claude_agent_sdk.types import StreamEvent
import asyncio

async def stream_response():
    options = ClaudeAgentOptions(
        include_partial_messages=True,
        allowed_tools=["Bash", "Read"],
    )

    async for message in query(prompt="List the files in my project",
options=options):
        if isinstance(message, StreamEvent):
            event = message.event
            if event.get("type") == "content_block_delta":
                delta = event.get("delta", {})
                if delta.get("type") == "text_delta":
                    print(delta.get("text", ""), end="", flush=True)

asyncio.run(stream_response())

```

StreamEvent reference

When partial messages are enabled, you receive raw Claude API streaming events wrapped in an object. The type has different names in each SDK:

- **Python:** `StreamEvent` (import from `claude_agent_sdk.types`)
- **TypeScript:** `SDKPartialAssistantMessage` with `type: 'stream_event'`

Both contain raw Claude API events, not accumulated text. You need to extract and accumulate text deltas yourself. Here's the structure of each type:

```

@dataclass
class StreamEvent:
    uuid: str # Unique identifier for this event
    session_id: str # Session identifier
    event: dict[str, Any] # The raw Claude API stream event
    parent_tool_use_id: str | None # Parent tool ID if from a subagent

```

The `event` field contains the raw streaming event from the Claude API. Common event types include:

Event Type	Description
<code>message_start</code>	Start of a new message
<code>content_block_start</code>	Start of a new content block (text or tool use)
<code>content_block_delta</code>	Incremental update to content
<code>content_block_stop</code>	End of a content block
<code>message_delta</code>	Message-level updates (stop reason, usage)
<code>message_stop</code>	End of the message

Message flow

With partial messages enabled, you receive messages in this order:

```
StreamEvent (message_start)
StreamEvent (content_block_start) - text block
StreamEvent (content_block_delta) - text chunks...
StreamEvent (content_block_stop)
StreamEvent (content_block_start) - tool_use block
StreamEvent (content_block_delta) - tool input chunks...
StreamEvent (content_block_stop)
StreamEvent (message_delta)
StreamEvent (message_stop)
AssistantMessage - complete message with all content
... tool executes ...
... more streaming events for next turn ...
ResultMessage - final result
```

Without partial messages enabled (`include_partial_messages` in Python, `includePartialMessages` in TypeScript), you receive all message types except `StreamEvent` . Common types include `SystemMessage` (session initialization), `AssistantMessage` (complete responses), `ResultMessage` (final result), and a compact boundary message indicating when conversation history was compacted (`SDKCompactBoundaryMessage` in TypeScript; `SystemMessage` with subtype `"compact_boundary"` in Python).

Stream text responses

To display text as it's generated, look for `content_block_delta` events where `delta.type` is `text_delta`. These contain the incremental text chunks. The example below prints each chunk as it arrives:

```
from claude_agent_sdk import query, ClaudeAgentOptions
from claude_agent_sdk.types import StreamEvent
import asyncio

async def stream_text():
    options = ClaudeAgentOptions(include_partial_messages=True)

    async for message in query(prompt="Explain how databases work",
options=options):
        if isinstance(message, StreamEvent):
            event = message.event
            if event.get("type") == "content_block_delta":
                delta = event.get("delta", {})
                if delta.get("type") == "text_delta":
                    # Print each text chunk as it arrives
                    print(delta.get("text", ""), end="", flush=True)

    print() # Final newline

asyncio.run(stream_text())
```

Stream tool calls

Tool calls also stream incrementally. You can track when tools start, receive their input as it's generated, and see when they complete. The example below tracks the current tool being called and accumulates the JSON input as it streams in. It uses three event types:

- `content_block_start` : tool begins
- `content_block_delta` with `input_json_delta` : input chunks arrive
- `content_block_stop` : tool call complete

```

from claude_agent_sdk import query, ClaudeAgentOptions
from claude_agent_sdk.types import StreamEvent
import asyncio

async def stream_tool_calls():
    options = ClaudeAgentOptions(
        include_partial_messages=True,
        allowed_tools=["Read", "Bash"],
    )

    # Track the current tool and accumulate its input JSON
    current_tool = None
    tool_input = ""

    async for message in query(prompt="Read the README.md file",
options=options):
        if isinstance(message, StreamEvent):
            event = message.event
            event_type = event.get("type")

            if event_type == "content_block_start":
                # New tool call is starting
                content_block = event.get("content_block", {})
                if content_block.get("type") == "tool_use":
                    current_tool = content_block.get("name")
                    tool_input = ""
                    print(f"Starting tool: {current_tool}")

            elif event_type == "content_block_delta":
                delta = event.get("delta", {})
                if delta.get("type") == "input_json_delta":
                    # Accumulate JSON input as it streams in
                    chunk = delta.get("partial_json", "")
                    tool_input += chunk
                    print(f"  Input chunk: {chunk}")

            elif event_type == "content_block_stop":
                # Tool call complete - show final input
                if current_tool:
                    print(f"Tool {current_tool} called with: {tool_input}")
                    current_tool = None

```

```
asyncio.run(stream_tool_calls())
```

Build a streaming UI

This example combines text and tool streaming into a cohesive UI. It tracks whether the agent is currently executing a tool (using an `in_tool` flag) to show status indicators like `[Using Read...]` while tools run. Text streams normally when not in a tool, and tool completion triggers a “done” message. This pattern is useful for chat interfaces that need to show progress during multi-step agent tasks.

```

from claude_agent_sdk import query, ClaudeAgentOptions, ResultMessage
from claude_agent_sdk.types import StreamEvent
import asyncio
import sys

async def streaming_ui():
    options = ClaudeAgentOptions(
        include_partial_messages=True,
        allowed_tools=["Read", "Bash", "Grep"],
    )

    # Track whether we're currently in a tool call
    in_tool = False

    async for message in query(
        prompt="Find all TODO comments in the codebase", options=options
    ):
        if isinstance(message, StreamEvent):
            event = message.event
            event_type = event.get("type")

            if event_type == "content_block_start":
                content_block = event.get("content_block", {})
                if content_block.get("type") == "tool_use":
                    # Tool call is starting - show status indicator
                    tool_name = content_block.get("name")
                    print(f"\n[Using {tool_name}...]", end="", flush=True)
                    in_tool = True

            elif event_type == "content_block_delta":
                delta = event.get("delta", {})
                # Only stream text when not executing a tool
                if delta.get("type") == "text_delta" and not in_tool:
                    sys.stdout.write(delta.get("text", ""))
                    sys.stdout.flush()

            elif event_type == "content_block_stop":
                if in_tool:
                    # Tool call finished
                    print(" done", flush=True)
                    in_tool = False

        elif isinstance(message, ResultMessage):

```

```
# Agent finished all work
print(f"\n\n--- Complete ---")
```

```
asyncio.run(streaming_ui())
```

Known limitations

- **Structured output:** the JSON result appears only in the final `ResultMessage.structured_output` , not as streaming deltas. See **structured outputs** for details.

Next steps

Now that you can stream text and tool calls in real-time, explore these related topics:

- **Interactive vs one-shot queries**: choose between input modes for your use case
- **Structured outputs**: get typed JSON responses from the agent
- **Permissions**: control which tools the agent can use